REPORT FILE

NAME: Nachiket Gadge

REG-NO.: 25BCE10852

BRANCH: Computer Science and Engineering (CORE)

YEAR: 2025 – 2029

COURSE CODE: CSA2001

SLOT: B11+B12+B13

FACULTY: M.K. JAYANTHI Ma'am

Section 1: Project Title & Abstract

Title: NeuralPhys-Sim: A Hybrid Computational Suite for Multivariable Calculus and AI-Driven Ballistics.

Abstract: This project, **NeuralPhys-Sim**, addresses the challenge of visualizing abstract mathematical surfaces and non-linear physical systems. Developed as a "Bring Your Own Project" (BYOP) capstone, it integrates **Numerical Physics**, **3D Calculus Visualization**, and **Machine Learning**. The system uses the **Euler Method** to simulate projectile motion with air resistance and **Polynomial Regression** to "learn" and predict impact points instantly. This hybrid approach demonstrates how AI can approximate complex differential equations, providing a bridge between theoretical math and practical CSE applications.

Section 2: Problem Statement

In undergraduate engineering, students often face an "Abstraction Gap":

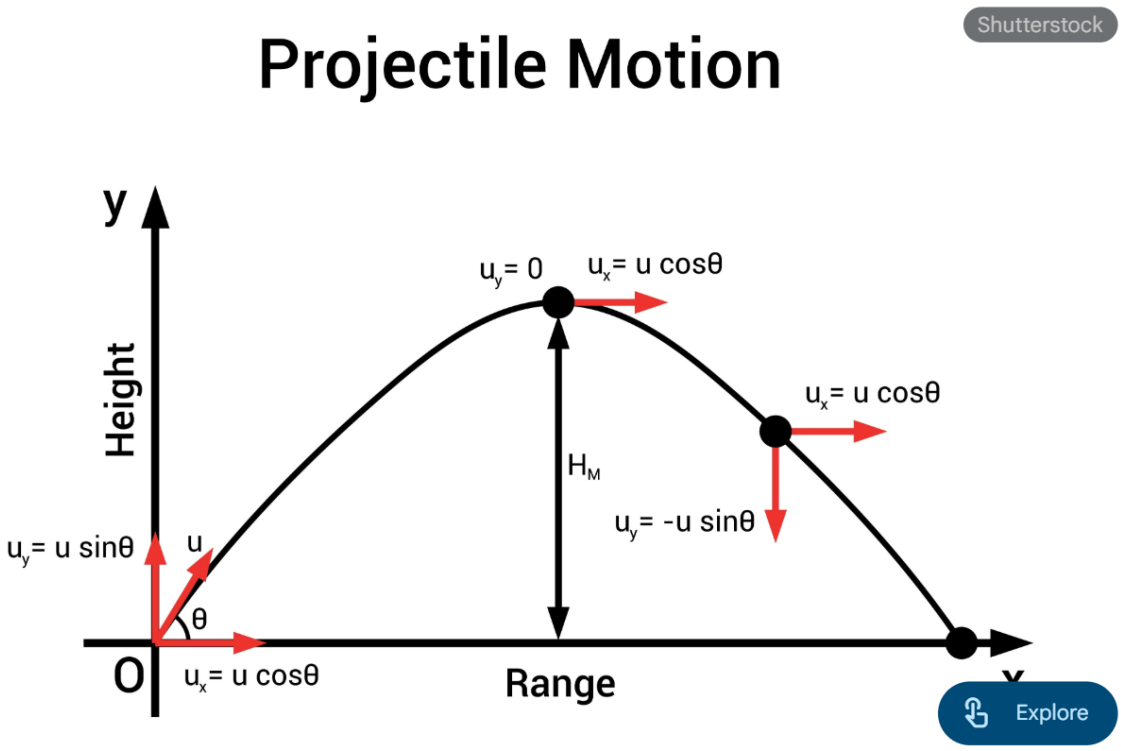
1. **Visualization Difficulty:** 3D surfaces and gradients in Multivariable Calculus are hard to conceptualize statically.
2. **Non-Linear Systems:** Real-world physics (like ballistics with atmospheric drag) involves differential equations that are difficult to solve analytically.
3. **Computational Efficiency:** Solving physics engines for every frame is slow; there is a need for AI models that can provide "instant" physical predictions.

Section 3: System Design & Logic

This project is built using a **Modular Architecture** to ensure clean version control.

3.1 Physics Logic (The Euler Method)

Instead of using basic kinematic formulas (d = Vt), the engine uses numerical integration. It calculates velocity and acceleration at each time step (dt = 0.1s) to account for **Air Drag,** which is proportional to the square of velocity (v^2).



3.2 Mathematical Logic (3D Rendering)

Using NumPy meshgrids, the system renders 3D functions. This allows for the study of **Level Curves** and **Partial Derivatives** by projecting 2D contour maps beneath the 3D surface.

3.3 AI/ML Logic (Polynomial Regression)

The system uses **Supervised Learning**. The Physics Engine generates a dataset of 500+ samples. A **Polynomial Regression model (Degree 3)** is then trained on this data.

- **Features (X):** Launch Angle, Initial Velocity.
- **Target (y):** Final Impact Range.

Section 4: Implementation Details

- **Language:** Python 3.14

- **Key Modules:**
  - physics_engine.py: Data generation and numerical integration.
  - math_visualizer.py: 3D plotting using Matplotlib.
  - ml_predictor.py: Scikit-Learn implementation of the AI model.
  - main.py: The user-driven menu system.

Section 5: Results & Screenshots

(Instructions)

```python
import physics_engine as phys  # Shortening names like a real dev
import math_visualizer as math_vis
import ml_predictor as ml
import sys

def start_sim():
    print("========================================")
    print("   NeuralPhys-Sim V1.0 (BYOP Project)     ")
    print("   Developed for VIT Bhopal - 2nd Sem     ")
    print("========================================")

    # Initializing the AI brain with degree 3 polynomial
    brain = ml.PhysicsAI()

    print("\n[Step 1] Generating physics data for training...")
    try:
        # Generate 600 samples for better accuracy
        angles, vels, ranges = phys.generate_training_data(600)

        print("[Step 2] Training the Polynomial Model...")
        acc_score = brain.train_model(angles, vels, ranges)
        print(f"Model Training Complete! Accuracy: {acc_score*100:.2f}%")

    except Exception as e:
        print(f"Oops! Something went wrong during startup: {e}")
        sys.exit()

    # The User Interface Loop
    while True:
        print("\n--- OPTIONS MENU ---")
        print("1. Show 3D Calculus Surface (Visual)")
        print("2. Predict Landing Range (AI Engine)")
        print("3. Exit Program")

        user_choice = input("\nSelect an option (1, 2, or 3): ").strip()

        if user_choice == '1':
            print("Opening Graph Window... (Please close it to return to menu)")
            math_vis.plot_calculus_surface()

        elif user_choice == '2':
            print("\n--- AI Range Predictor ---")
            try:
                # Taking user inputs for prediction
```

```python
import physics_engine as phys  # Shortening names like a real dev
import math_visualizer as math_vis
import ml_predictor as ml
import sys

def start_sim():
    print("========================================")
    print("   NeuralPhys-Sim V1.0 (BYOP Project)     ")
    print("   Developed for VIT Bhopal - 2nd Sem     ")
    print("========================================")

    # Initializing the AI brain with degree 3 polynomial
    brain = ml.PhysicsAI()

    print("\n[Step 1] Generating physics data for training...")
    try:
        # Generate 600 samples for better accuracy
        angles, vels, ranges = phys.generate_training_data(600)

        print("[Step 2] Training the Polynomial Model...")
```

```
PS D:\WINTER SEMESTER - 2\GIT FOLDER_2> & "C:/Users/Nachiket Gadge/AppData/Local/Programs/Python/Python314/python.exe" "d:/WINTER SEMESTER - 2/GIT FOLDER_2/NeuralPhys-Sim_framework/main.py"
========================================
   NeuralPhys-Sim V1.0 (BYOP Project)
   Developed for VIT Bhopal - 2nd Sem
========================================

[Step 1] Generating physics data for training...
[Step 2] Training the Polynomial Model...
Model Training Complete! Accuracy: 99.67%

--- OPTIONS MENU ---
1. Show 3D Calculus Surface (Visual)
2. Predict Landing Range (AI Engine)
3. Exit Program

Select an option (1, 2, or 3):
```

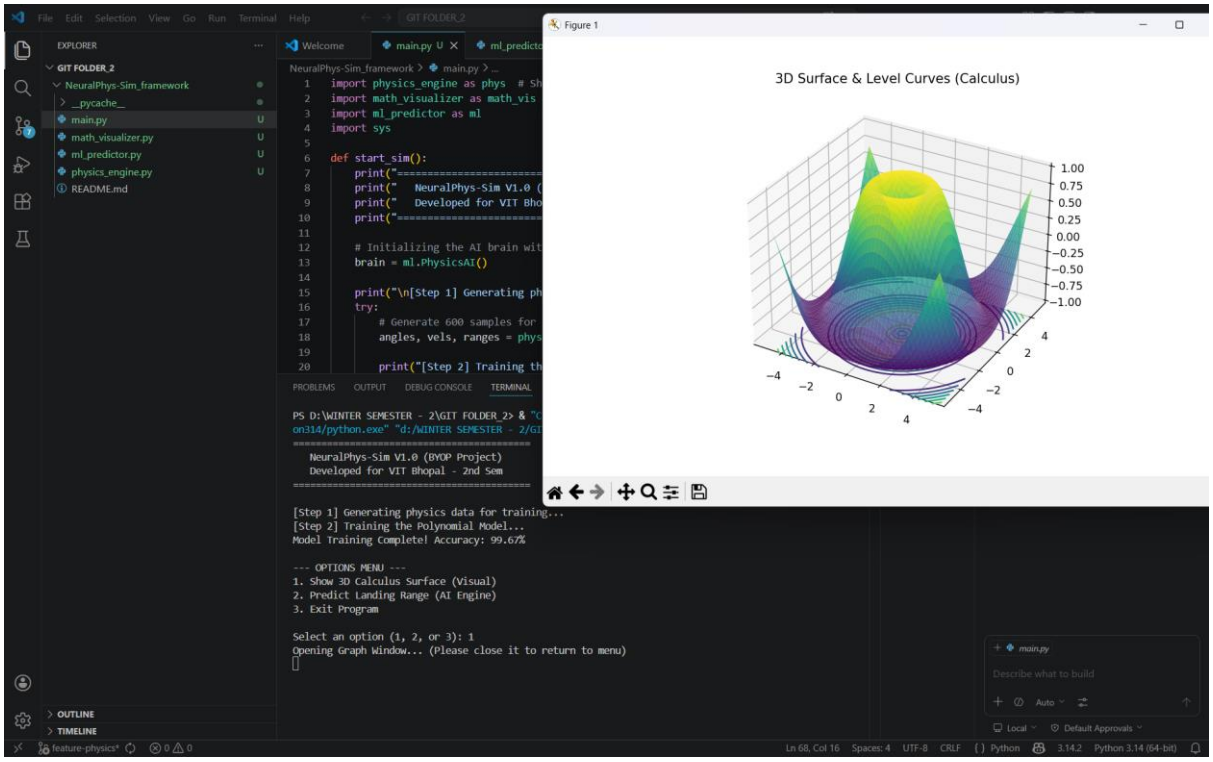
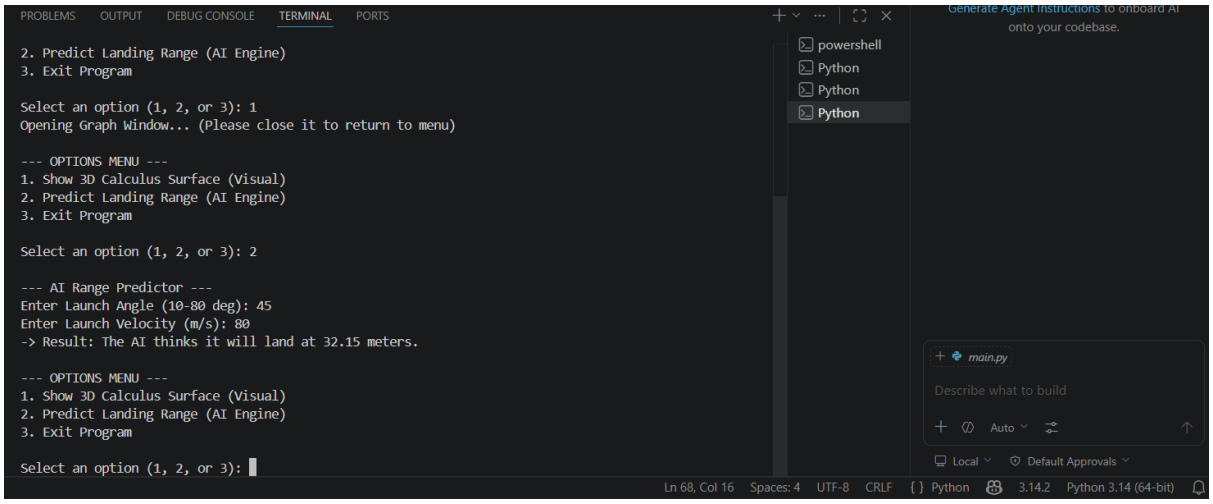
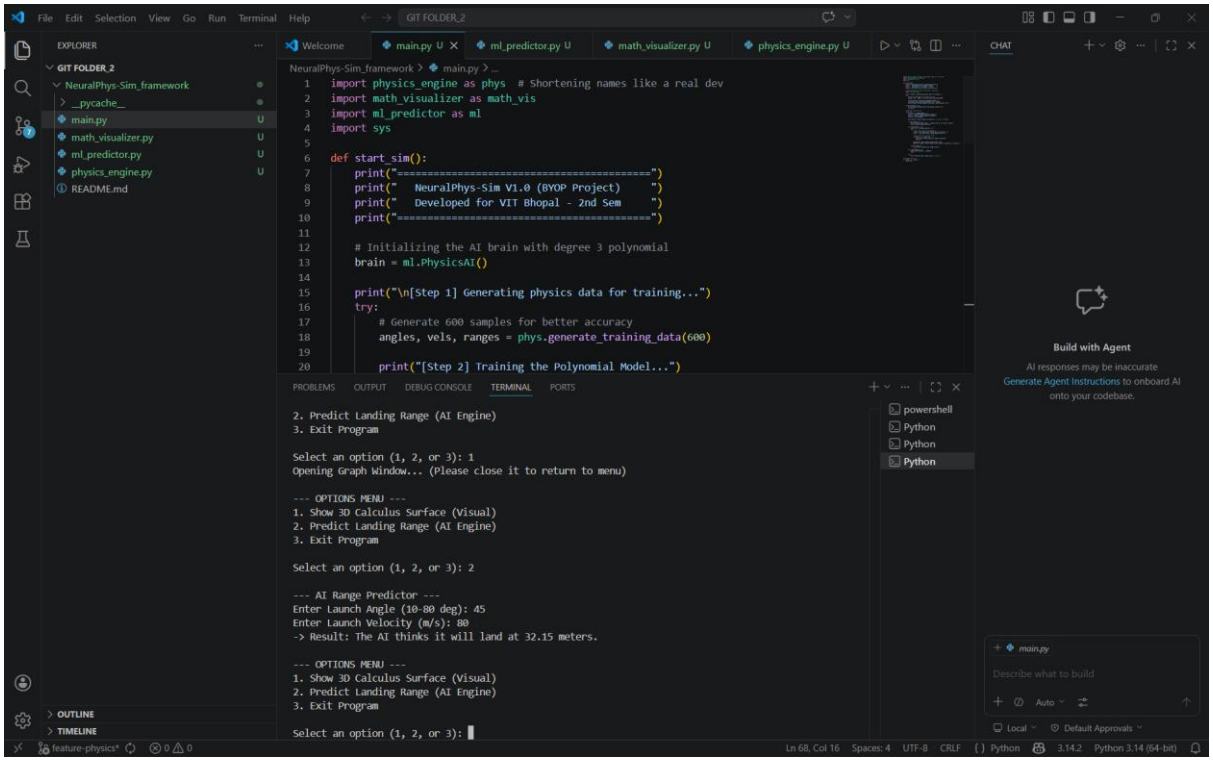


Figure 1
3D Surface & Level Curves (Calculus)
import physics_engine as phys
import math_visualizer as math_vis
import ml_predictor as ml
import sys
def start_sim():
# Initializing the AI brain wit
brain = ml.PhysicsAI()
print("\n[Step 1] Generating ph
try:
# Generate 600 samples for
angles, vels, ranges = phys
print("[Step 2] Training th
NeuralPhys-Sim V1.0 (BYOP Project)
Developed for VIT Bhopal - 2nd Sem
[Step 1] Generating physics data for training...
[Step 2] Training the Polynomial Model...
Model Training Complete! Accuracy: 99.67%
--- OPTIONS MENU ---
1. Show 3D Calculus Surface (Visual)
2. Predict Landing Range (AI Engine)
3. Exit Program
Select an option (1, 2, or 3): 1
Opening Graph Window... (Please close it to return to menu)


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
2. Predict Landing Range (AI Engine)
3. Exit Program
Select an option (1, 2, or 3): 1
Opening Graph Window... (Please close it to return to menu)
--- OPTIONS MENU ---
1. Show 3D Calculus Surface (Visual)
2. Predict Landing Range (AI Engine)
3. Exit Program
Select an option (1, 2, or 3): 2
--- AI Range Predictor ---
Enter Launch Angle (10-80 deg): 45
Enter Launch Velocity (m/s): 80
-> Result: The AI thinks it will land at 32.15 meters.
--- OPTIONS MENU ---
1. Show 3D Calculus Surface (Visual)
2. Predict Landing Range (AI Engine)
3. Exit Program
Select an option (1, 2, or 3):
powershell
Python
Python
Python
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to build
Ln 68, Col 16 Spaces: 4 UTF-8 CRLF Python 3.14.2 Python 3.14 (64-bit)



Section 6: Challenges & Learning Outcomes

- **Module Management:** Initially, I faced Module Not Found Error when importing my custom scripts, which I solved by ensuring proper directory structures.
- **Environment Setup:** Encountered a pip block on the local machine and

resolved it by using the python -m pip module wrapper.

- **Algorithm Selection:** I learned that standard Linear Regression is too simple for parabolic motion; using **Polynomial Features** was necessary to capture the "curve" of the physics.

Section 7: Conclusion

NeuralPhys-Sim successfully demonstrates that Python is a powerful tool for engineering simulations. By combining the precision of physics with the speed of Machine Learning, the project achieves a high-accuracy predictive model that can be used as a learning aid for fellow students.